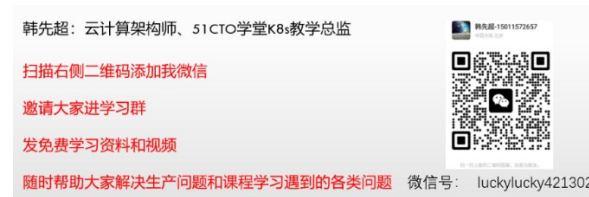


我是 51CTO 学堂 k8s 和 python 教学总监韩先超，清华大学出版社出版的【Kubernetes 从入门到 DevOps 企业应用实战】畅销书作者，大家百度搜索韩先超，可以看到我详细的介绍，我录制的课程定位是既能学到技术本身，又能用于生产环境，课程内容都是结合公司实际项目讲的，我接触的项目很多，也去过国企，大公司做过架构设计（电信、移动、上海咪咕视讯、天翼物联、深圳任子行等），也对很多公司生产故障进行过现场排查解决，所以这些年积累的经验和实战项目，都会放到我课程里，希望大家跟我一起学习，少走弯路，多挣钱，不断提升自己~

备注：韩先超老师社交平台，建议都关注下，会定期更新不同方向的技术文章和视频：

1、韩先超老师个人微信：luckylucky421302

也可以扫描下方二维码添加：



2、韩先超老师微信公众号：定期发布技术类文章（运维、开发、测试、DBA、网工、渗透安全、K8S 等均覆盖）



3、扫面下面二维码，关注韩老师视频号（定期发布免费技术视频，随时观看）：



4、韩老师抖音号：打开抖音搜索 hxck8s1，定期开直播，发布免费视频，免费分享技术：如 Linux、Docker、K8S、DBA、安全渗透、网路等，扫描下方二维码也可以关注：



5、韩老师 b 站账号: bili_55093353115



使用 Tkinter 创建一个 GUI（用户图形界面）应用程序，用于管理 Kubernetes 中的 Deployment 和 StatefulSet。用户可以选择命名空间，选择资源（Deployment 或 StatefulSet），并更新其副本数。

tkinter 是 Python 的标准 GUI（图形用户界面）库，它提供了创建窗口、按钮、文本框、菜单等 GUI 组件的功能。tkinter 是一个跨平台的工具，可以在 Windows、macOS 和 Linux 等操作系统上运行，使得开发者可以方便地创建桌面应用程序。

主要特点

1. **易于使用**：tkinter 提供了简单的 API，适合初学者和希望快速开发 GUI 应用程序的开发者。
2. **跨平台**：应用程序可以在不同操作系统上运行，提供一致的用户体验。
3. **丰富的组件**：支持多种 GUI 组件，如按钮、标签、文本框、菜单、框架、画布等，能够满足大多数桌面应用的需求。
4. **支持事件驱动编程**：可以轻松处理用户输入和事件，例如鼠标点击、键盘输入等。
5. **可扩展性**：虽然 tkinter 是一个轻量级的库，但它也允许开发者通过其他 Python 库进行扩展。

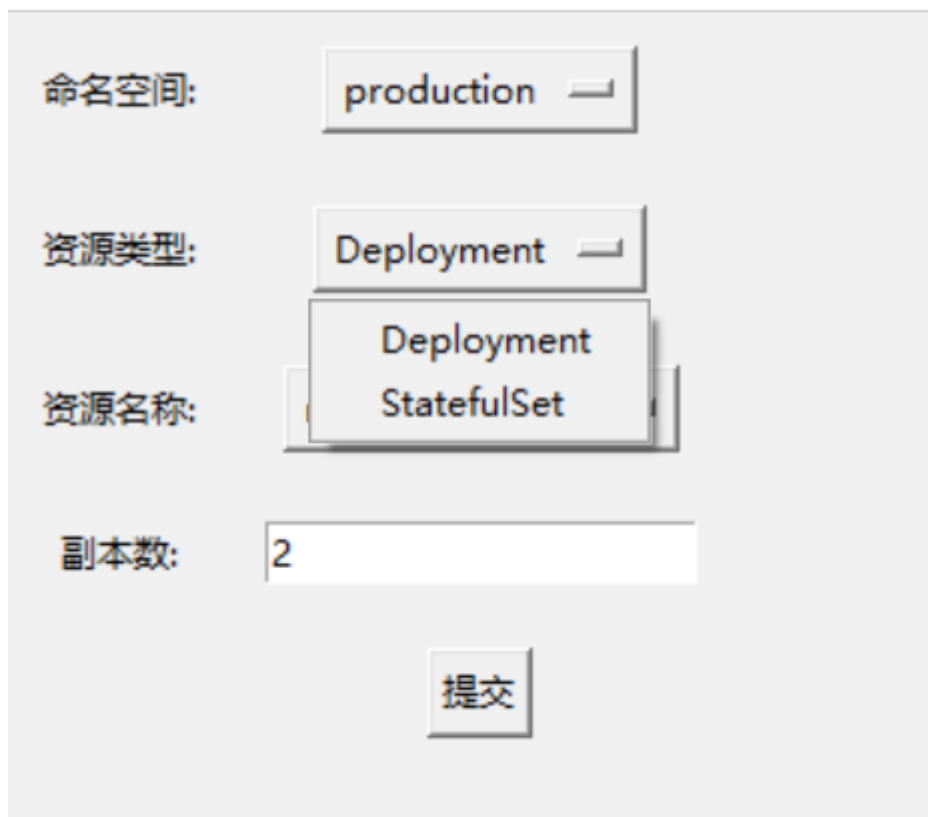
基本组件

- 窗口 (Tk): 应用程序的主窗口。
- 标签 (Label): 用于显示文本或图像。
- 按钮 (Button): 可点击的按钮。
- 文本框 (Entry 和 Text): 用于单行或多行文本输入。
- 框架 (Frame): 用于组织和布局其他组件。
- 菜单 (Menu): 用于创建应用程序的菜单栏。

 Kubernetes Deployment 和 StatefulSet 管理

命名空间:	production
资源类型:	Deployment
资源名称:	my-deployment
副本数:	2
提交	

🔗 Kubernetes Deployment 和 StatefulSet 管理



命名空间: production

资源类型: Deployment

资源名称: Deployment
StatefulSet

副本数: 2

提交

具体功能：

- 1、从 k8s 集群中选择需要操作的命名空间。
- 2、选择资源类型（Deployment 和 StatefulSet）。
- 3、选择资源名称。
- 4、输入需要修改的副本数并提交。

实现方式：

通过 python 脚本开发一个 GUI，并封装成 windows 程序，能够通过点击图标启动 gui 界面，可以实现修改 k8s 集群 deployment 和 statefulset 的 pod 副本数。

1、代码：k8s-repli.py

```
import tkinter as tk
from tkinter import messagebox
from kubernetes import client, config

# 加载 Kubernetes 配置
config.load_kube_config(config_file='D:/config')

# 创建 API 对象
```

```

apps_v1 = client.AppsV1Api()
core_v1 = client.CoreV1Api()

# 获取所有命名空间
def get_namespaces():
    return [ns.metadata.name for ns in core_v1.list_namespace().items]

# 获取所有 Deployment 和 StatefulSet
def get_resources(namespace):
    deployments = apps_v1.list_namespaced_deployment(namespace=namespace).items
    statefulsets = apps_v1.list_namespaced_stateful_set(namespace=namespace).items
    resources = [("Deployment", d.metadata.name) for d in deployments] + [("StatefulSet",
s.metadata.name) for s in statefulsets]
    return resources

def update_deployment_or_statefulset():
    namespace = namespace_var.get().strip()
    resource_type = resource_type_var.get().strip() # 获取资源类型 (Deployment 或
StatefulSet)
    resource_name = resource_var.get().strip() # 获取资源名称
    replicas = replicas_entry.get().strip()

    try:
        replicas = int(replicas)
    except ValueError:
        messagebox.showerror("副本数无效", "请输入有效的整数作为副本数。")
        return

    try:
        if resource_type == "Deployment":
            # 更新 Deployment 副本数
            deployment = apps_v1.read_namespaced_deployment(name=resource_name,
namespace=namespace)
            deployment.spec.replicas = replicas
            apps_v1.patch_namespaced_deployment(name=resource_name,
namespace=namespace, body=deployment)
            messagebox.showinfo("更新成功", f"Deployment '{resource_name}' 在命名空间
'{namespace}' 中已更新, 副本数为 {replicas}。")
        elif resource_type == "StatefulSet":
            # 更新 StatefulSet 副本数
            statefulset = apps_v1.read_namespaced_stateful_set(name=resource_name,
namespace=namespace)
            statefulset.spec.replicas = replicas
            apps_v1.patch_namespaced_stateful_set(name=resource_name,

```

```

namespace=namespace, body=statefulset)
    messagebox.showinfo("更新成功", f"StatefulSet '{resource_name}' 在命名空间
'{namespace}' 中已更新，副本数为 {replicas}。")
except client.ApiException as e:
    messagebox.showerror("错误", f"发生错误: {e.reason}")

# 更新资源名称下拉菜单
def update_resources_menu(*args):
    namespace = namespace_var.get()
    resources = get_resources(namespace)
    resource_menu['menu'].delete(0, 'end')
    for resource_type, resource_name in resources:
        resource_menu['menu'].add_command(label=resource_name,
command=tk._setit(resource_var, resource_name))
    # 默认选择第一个资源
    if resources:
        resource_var.set(resources[0][1])
    else:
        resource_var.set("")

# 创建 GUI 窗口
root = tk.Tk()
root.title("Kubernetes Deployment 和 StatefulSet 管理")

# 获取命名空间
valid_namespaces = get_namespaces()

# 命名空间选择下拉菜单
tk.Label(root, text="命名空间:").grid(row=0, column=0, padx=10, pady=10)
namespace_var = tk.StringVar(root)
namespace_var.set(valid_namespaces[0]) # 设置默认值
namespace_menu = tk.OptionMenu(root, namespace_var, *valid_namespaces)
namespace_menu.grid(row=0, column=1, padx=10, pady=10)

# 资源类型选择
tk.Label(root, text="资源类型:").grid(row=1, column=0, padx=10, pady=10)
resource_type_var = tk.StringVar(root)
resource_type_var.set("Deployment") # 默认选择 Deployment
resource_type_menu = tk.OptionMenu(root, resource_type_var, "Deployment", "StatefulSet")
resource_type_menu.grid(row=1, column=1, padx=10, pady=10)

# 资源名称选择下拉菜单
tk.Label(root, text="资源名称:").grid(row=2, column=0, padx=10, pady=10)
resource_var = tk.StringVar(root)

```

```

resource_menu = tk.OptionMenu(root, resource_var, "")
resource_menu.grid(row=2, column=1, padx=10, pady=10)

# 当命名空间变化时更新资源名称下拉菜单
namespace_var.trace_add('write', update_resources_menu)

# 副本数输入
tk.Label(root, text="副本数:").grid(row=3, column=0, padx=10, pady=10)
replicas_entry = tk.Entry(root)
replicas_entry.grid(row=3, column=1, padx=10, pady=10)

# 提交按钮
submit_button = tk.Button(root, text="提交", command=update_deployment_or_statefulset)
submit_button.grid(row=4, column=1, padx=10, pady=10)

# 运行窗口
update_resources_menu() # 初始更新资源名称列表
root.mainloop()

```

2. 打包成 Windows 可执行文件 (.exe)

为了将该 Python 脚本打包成一个 Windows 可执行文件，可以使用 pyinstaller 工具。具体步骤如下：

2.1 安装 pyinstaller

首先需要安装 pyinstaller，可以通过以下命令安装：

```
pip3 install pyinstaller
```

2.2 打包成 .exe

进入你的脚本目录，然后运行以下命令打包为可执行文件：

```
pyinstaller --onefile --windowed .\k8s-repli.py
```

- --onefile: 将所有依赖打包成一个单独的文件。
- --windowed: 不显示命令行窗口（适用于 GUI 程序）。

2.3 生成的文件

打包完成后，你会在 dist 文件夹下找到生成的 .exe 文件。点击这个文件即可启动程序，显示输入框来输入命名空间和副本数。

总结

通过使用 Tkinter 你可以创建一个简单的 GUI 界面，用户可以通过该界面输入所需的参数。随后使用 pyinstaller 将其打包为 .exe 文件，方便在 Windows 系统中使用。

#代码解读：

该脚本是一个基于 Tkinter 的图形用户界面 (GUI) 程序，允许用户通过图形界面操作 Kubernetes 集群中的 Deployment 和 StatefulSet。用户可以选择命名空间，选择需要操作的资源，并更新其副本数 (replicas)。脚本使用了 Kubernetes Python 客户端 (kubernetes-python-client) 来与 Kubernetes API 进行交互，从而修改集群中资源的配置。

1. **代码详细解释**:

- ``import tkinter as tk``: 导入 Tkinter 模块，用于创建图形界面，Tkinter 是 Python 标准库的一部分，用于构建桌面应用程序的用户界面。

- ``from tkinter import messagebox``: 从 Tkinter 导入 messagebox，用于弹出对话框，显示错误或提示信息。

- ``from kubernetes import client, config``: 从 Kubernetes 客户端模块中导入 client 和 config。`client` 提供了与 Kubernetes API 交互的功能，而 `config` 用于加载 Kubernetes 配置文件 (kubeconfig)。

- ``config.load_kube_config(config_file='D:/config')``: 通过指定的文件路径加载 Kubernetes 配置文件，用于认证和与 Kubernetes 集群通信。`D:/config` 是配置文件的路径，里面包含了连接集群所需的凭据和配置信息。

2. **API 对象的创建**:

- ``apps_v1 = client.AppsV1Api()``: 创建了 `AppsV1Api` 类的实例，用于与 Kubernetes 集群中的 `Deployment` 和 `StatefulSet` 资源进行交互。这个对象提供了管理这些资源的 API 方法，比如获取、更新 Deployment 和 StatefulSet。

- ``core_v1 = client.CoreV1Api()``: 创建了 `CoreV1Api` 类的实例，用于与 Kubernetes 集群的核心资源进行交互，例如命名空间 (namespace)。通过该对象可以获取、创建或删除命名空间。

3. **函数解释**:

- ``def get_namespaces()``: 获取 k8s 集群中所有命名空间的函数。

- 1) `core_v1.list_namespace()` 调用了 Kubernetes API 来获取集群中的所有命名空间，返回的是一个包含命名空间信息的对象。

- 2) `items` 是一个列表，包含了每个命名空间的详细信息 (如名称、状态、创建时间等)。

- 3) `[ns.metadata.name for ns in core_v1.list_namespace().items]` 是列表推导式，用来遍历 `items` 中的每个命名空间对象 `ns`，提取每个命名空间对象中的 `metadata.name` 属性，也就是命名空间的名称。

最终的结果是一个由所有命名空间名称组成的列表。例如，如果集群中有两个命名空间 `default` 和 `kube-system`，结果将是：

```
["default", "kube-system"]
```

- ``def get_resources(namespace)``: 获取指定命名空间下所有 Deployment 和 StatefulSet 的函数。

1. ``apps_v1.list_namespaced_deployment(namespace).items`` 方法用于列出指定命名空间中的所有 Deployment 资源，放到列表里。

2. ``apps_v1.list_namespaced_stateful_set(namespace).items`` 方法用于列出指定命名空间中的所有 StatefulSet 资源，放到列表里。

3. 通过将 Deployment 和 StatefulSet 资源分别存储为元组 (type, name) 的形式，将它们组合到一个列表中返回：

```
```python
resources = [("Deployment", d.metadata.name) for d in deployments] + [("StatefulSet",
s.metadata.name) for s in statefulsets]
return resources
```
```

4. **副本数更新的核心逻辑**:

- ``def update_deployment_or_statefulset()``: 这个函数用于根据用户选择的资源类型和名称来更新 Deployment 或 StatefulSet 的副本数。

1. ``namespace_var.get()``: 获取 gui 界面中用户在命名空间下拉菜单中选择的命名空间。

2. ``resource_type_var.get()``: 获取 gui 界面中用户在资源类型下拉菜单中选择的资源类型

3. ``resource_var.get()``: 获取 gui 界面中用户在资源名称下拉菜单中选择的具体资源名称。

4. ``replicas_entry.get()``: 获取用户输入的副本数量，并验证它是否是一个有效的整数。如果输入无效，将弹出错误消息。

根据用户选择的资源类型，调用相应的 Kubernetes API 来更新副本数：

- 如果资源类型是 ``Deployment``，则调用 ``apps_v1.read_namespaced_deployment()`` 获取 Deployment 对象，然后更新其 ``spec.replicas`` 属性，并通过 ``patch_namespaced_deployment()`` 方法将更改提交给 Kubernetes 集群。

- 如果资源类型是 ``StatefulSet``，则类似地，通过 ``apps_v1.read_namespaced_stateful_set()`` 获取 StatefulSet 对象，并更新其副本数。

更新成功后，弹出信息框提示用户操作结果。

5. **资源下拉菜单更新**:

- ``def update_resources_menu(*args)``: 当用户选择命名空间时，自动调用此函数来更新资源下拉菜单，以列出当前命名空间中的所有 Deployment 和 StatefulSet。

1. 调用 ``get_resources(namespace)`` 函数获取当前命名空间下的所有资源。

2. 使用 Tkinter 的 ``OptionMenu`` 组件更新菜单项，确保用户能够选择新命名空间下的资源。

3. 如果新命名空间下没有资源，资源菜单将显示为空。

7. **Tkinter 图形界面布局**:

- `root = tk.Tk()`: 创建主窗口 `root`
 `tk.Tk()`: 创建一个顶层窗口对象 `root`, 这是整个图形界面的主窗口。
 `root.title("Kubernetes Deployment 和 StatefulSet 管理")`: 设置窗口的标题为“Kubernetes Deployment 和 StatefulSet 管理”。

获取命名空间

`valid_namespaces = get_namespaces()`

创建一个带有标签和下拉菜单的界面元素, 让用户可以选择 **Kubernetes** 的命名空间。

`tk.Label(root, text="命名空间:").grid(row=0, column=0, padx=10, pady=10)`

`tk.Label(root, text="命名空间:")`: 这是一个标签控件 (Label), 用于在窗口中显示提示文本“命名空间:”, 告诉用户他们在此选择命名空间。

- **`root`**: 表示这个标签属于主窗口 `root`。
- **`text="命名空间:"`**: 定义标签显示的文本为 “命名空间:”。
- **`.grid(row=0, column=0, padx=10, pady=10)`**: 这个部分定义了标签的位置和布局。
- **`row=0, column=0`**: 表示标签位于窗口的第 0 行、第 0 列。
- **`padx=10, pady=10`**: 为标签周围增加 10 像素的水平和垂直间距, 使其看起来更加美观。

#创建绑定变量 **`namespace_var`**:

`namespace_var = tk.StringVar(root)`

`tk.StringVar(root)`: `StringVar` 是一个 Tkinter 提供的变量类型, 专门用于保存字符串, 并且它可以与图形界面的控件进行绑定。用户在界面中选择或输入的内容可以自动存储到这个变量中。

- **`namespace_var`** 用来存储用户从下拉菜单中选择的命名空间。
- **`root`** 表示这个变量与主窗口 `root` 绑定。

#设置默认命名空间:

`namespace_var.set(valid_namespaces[0])`

将下拉菜单的默认值设置为第一个命名空间。

- **`valid_namespaces`** 是之前获取的命名空间列表, `valid_namespaces[0]` 是这个列表中的第一个命名空间。
- **`.set()`** 方法会将 `namespace_var` 变量的值设置为这个默认的命名空间。

#创建命名空间下拉菜单 **`OptionMenu`**:

`namespace_menu = tk.OptionMenu(root, namespace_var, *valid_namespaces)`

`tk.OptionMenu(root, namespace_var, *valid_namespaces)`: 这是一个下拉菜单控件, 用于展示多个命名空间供用户选择。

- **`root`**: 表示下拉菜单属于主窗口 `root`。
- **`namespace_var`**: 下拉菜单与 `namespace_var` 变量绑定。当用户选择某个命名空间时, 这个变量的值会自动更新。
- **`*valid_namespaces`**: 传入命名空间列表 `valid_namespaces`, 并使用 `*` 运算符将其解包, 使得每一个命名空间成为下拉菜单中的一个选项

#布局下拉菜单:

```
namespace_menu.grid(row=0, column=1, padx=10, pady=10)
```

.grid(row=0, column=1): 使用 grid 布局将下拉菜单放置在窗口的第 0 行、第 1 列, 紧邻标签“命名空间:”的右侧。

padx=10, pady=10: 为下拉菜单周围增加 10 像素的内边距, 保持界面美观。

#在用户选择或更改命名空间时, 自动更新资源下拉菜单中的选项

1. namespace_var.trace_add('write', update_resources_menu) 的作用:

- **namespace_var.trace('write', update_resources_menu)** 是一个跟踪机制, 意味着当 namespace_var 这个变量的值发生变化时, 会触发 update_resources_menu 函数。

2. trace() 方法的详细解释:

- **trace()** 是 Tkinter 的 StringVar、IntVar 等变量类型提供的方法, 用于跟踪变量值的变化。
- **'w'** 是触发条件, 表示当变量的值被“写入”(即发生变化)时, 调用指定的回调函数。
 - **'w'**: 写入时触发 (即当变量的值发生变化时)。
 - **'r'**: 读取时触发 (当变量被访问时)。
 - **'u'**: 被删除时触发 (当变量被销毁时)。

3. 触发的回调函数 update_resources_menu:

- 当命名空间下拉菜单中的值 (即 namespace_var) 被用户修改时, 会调用 **update_resources_menu** 函数。
 - **update_resources_menu()** 函数的作用是获取新选择的命名空间, 获取该命名空间下的所有资源 (如 Deployment 和 StatefulSet), 并更新资源名称的下拉菜单。

4. 如何理解这段代码的流程:

- 当用户从命名空间下拉菜单中选择一个新的命名空间时, namespace_var 的值发生变化, trace('w', ...) 机制检测到这个变化, 并立即调用 update_resources_menu() 函数。
- **update_resources_menu()** 函数会重新获取该命名空间下的所有可用资源, 并将这些资源更新到资源名称的下拉菜单中, 供用户选择。

更新资源下拉菜单

```
def update_resources_menu(*args):
    namespace = namespace_var.get()
    resources = get_resources(namespace)
    resource_menu['menu'].delete(0, 'end')
    for resource_type, resource_name in resources:
        resource_menu['menu'].add_command(label=resource_name,
        command=tk._setit(resource_var, resource_name))
    # 默认选择第一个资源
    if resources:
        resource_var.set(resources[0][1])
    else:
```

```
resource_var.set("")
```

1) . update_resources_menu(*args) 函数:

- 这是一个回调函数，用来更新资源名称下拉菜单。它在命名空间发生变化时被调用（通过之前的 namespace_var.trace('w', update_resources_menu) 触发）。
- *args 是可变参数，用于接受跟踪机制传递的附加参数（例如，跟踪变量的名称、旧值和新值），但这里没有使用这些参数。

2) . namespace = namespace_var.get():

- 从 namespace_var 中获取当前选择的命名空间的值。namespace_var 是一个 StringVar 对象，存储了下拉菜单中用户选择的命名空间。

3) . resources = get_resources(namespace):

- 调用 get_resources(namespace) 函数，获取该命名空间下的所有资源，包括 Deployment 和 StatefulSet。
- 返回的 resources 是一个列表，格式为：[("Deployment", "deployment_name1"), ("StatefulSet", "statefulset_name1")], 其中每个元素是一个元组，包含资源类型和资源名称。

4) . resource_menu['menu'].delete(0, 'end'):

- 这行代码清空当前资源下拉菜单中的所有选项。
- resource_menu['menu'] 访问的是 OptionMenu 控件中的菜单部分，delete(0, 'end') 表示删除菜单中的所有选项。

5) . for resource_type, resource_name in resources::

- 这一行开始一个循环，遍历所有从 get_resources(namespace) 获取的资源，每个资源的类型 (resource_type) 和名称 (resource_name) 都会分别处理。

6) . resource_menu['menu'].add_command(...):

- 使用 add_command() 方法将新的资源选项添加到资源下拉菜单中。
- label=resource_name 指定了每个选项在下拉菜单中的显示名称（即资源名称）。
- command=tk.setit(resource_var, resource_name) 设定了当用户选择某个资源时，resource_var 的值会被设置为相应的 resource_name。
 - tk.setit(resource_var, resource_name) 是一种快速设置 Tkinter 变量值的方式，当用户选择资源时，资源名称会赋值给 resource_var。

7) . if resources::

- 检查 resources 列表是否非空（即是否有可用的资源）。如果有资源：
 - resource_var.set(resources[0][1]): 将第一个资源的名称 (resources[0][1]) 设置为默认选择的资源。resources[0][1] 代表第一个资源的名称。

8) . else::

- 如果 resources 列表为空（即当前命名空间中没有 Deployment 或 StatefulSet），执行以下操作：
 - resource_var.set(""): 将 resource_var 设置为空字符串，表示没有可用的资源可供选择。

#创建标签:

```
tk.Label(root, text="副本数:").grid(row=3, column=0, padx=10, pady=10)
```

tk.Label(root, text="副本数:") 创建了一个标签，显示的文字是 "副本数:"，用于提示用户输入副本数。这个标签会放置在窗口中。

.grid(row=3, column=0, padx=10, pady=10) 使用网格布局 (grid) 来指定标签的位置。它放在 **第 3 行, 第 0 列**, padx=10 和 pady=10 表示在标签周围留出 10 个像素的水平和垂直间距。

#创建输入框

```
replicas_entry = tk.Entry(root)
```

```
replicas_entry.grid(row=3, column=1, padx=10, pady=10)
```

tk.Entry(root) 创建了一个输入框对象, 名为 replicas_entry, 用户可以在这个输入框中输入副本数的数值。

.grid(row=3, column=1, padx=10, pady=10) 同样使用网格布局将输入框放置在窗口中, 位置是 **第 3 行, 第 1 列**, 和上面的标签排成一行。

padx=10, pady=10 用于给输入框设置水平和垂直间距。

创建提交按钮

```
submit_button = tk.Button(root, text="提交", command=update_deployment_or_statefulset)
```

```
submit_button.grid(row=4, column=1, padx=10, pady=10)
```

```
submit_button = tk.Button(root, text="提交",  
command=update_deployment_or_statefulset):
```

- tk.Button() 用于在界面上创建一个按钮。
- root 是按钮所属的窗口, 表示该按钮属于主窗口。
- text="提交": 按钮上显示的文字是"提交", 用来告诉用户点击此按钮后可以提交操作。
- command=update_deployment_or_statefulset: 这个参数指定了当用户点击按钮时要执行的函数。在这里, update_deployment_or_statefulset 是用于处理副本数更新操作的函数, 当用户点击 "提交" 按钮时, 它会被触发, 负责获取用户输入的副本数、命名空间和资源名称, 更新 Kubernetes 的 Deployment 或 StatefulSet。

```
submit_button.grid(row=4, column=1, padx=10, pady=10):
```

- 使用 grid() 方法来将按钮放置在窗口的网格布局中。
- row=4, column=1 表示按钮被放置在 **第 4 行, 第 1 列**。
- padx=10 和 pady=10 用于设置按钮与其他组件之间的水平和垂直间距。

初始更新资源列表

```
update_resources_menu()
```

这行代码在窗口运行前, 调用了 update_resources_menu() 函数。

它的目的是在程序启动时就 **根据默认命名空间填充资源列表** (如 Deployment 或 StatefulSet 的名称), 从而确保用户在第一次操作界面时能看到可选的资源。

不调用此函数的话, 资源下拉菜单一开始会是空的, 用户可能看不到可选择的资源。

#运行窗口的事件循环

```
root.mainloop()
```

root.mainloop() 是 Tkinter 中启动 **事件循环** 的方法。

事件循环的作用是使窗口保持在屏幕上并响应用户的输入 (如点击按钮、选择下拉菜单、输入文本等)。

当用户进行任何操作 (如点击提交按钮、选择命名空间等), 事件循环会处理这些事件,

并调用相应的回调函数（如 `update_deployment_or_statefulset()`）。

没有 `mainloop()`，窗口会一闪而过，用户将看不到图形界面。

总结：

- **提交按钮**：用于触发 `update_deployment_or_statefulset` 函数，用户点击后执行副本数更新操作。
- **初始化资源列表**：在程序启动时，调用 `update_resources_menu` 函数填充资源下拉菜单。
- **事件循环**：`root.mainloop()` 保持窗口在屏幕上，等待并响应用户的操作。